

Main program (Starts the simulation by launching the controller that manages everything.)

```
Algorithm Main
    Create ServiceCentreSimulation object
    Call run()
```

ServiceCentreSimulation Controller (Displays menu, reads user choice, routes to correct operation.)

```
Algorithm run()
    running <- true
    While running do
        printMenu()
        choice <- read user input
        Switch choice:
            Case 1: enqueueStudents()
            Case 2: serveStudent()
            Case 3: queue.displayQueue()
            Case 4: insertRecord()
            Case 5: records.displayStudents()
            Case 6: searchRecord()
            Case 7: deleteRecord()
            Case 8: Statistics.display(serviceTimes())
            Case 9: sortTimes()
            Case 10: sortingExperiment()
            Case 11: postfixDemo()
            Case 12: addManualStudent()
            Case 13: updateRecordServiceType()
            Case 14: insertRecordAtPosition()
            Case 0: running <- false
            Default: print "Invalid option"
    Print "Program ended"
```

Queue Operations (Manage the waiting line of students by enqueueing, serving (dequeuing), real-time countdown, tracking)

```
Algorithm enqueueStudents()
    For each sample student do
        If student not already in queue then
            queue.enqueue(student)
            records.insertStudentIfAbsent(student)
    remainingCount <- queue.size()
    Print number added and queue length
```

```
Algorithm serveStudent()
    Print "Choose dequeue option: 1 instant, 2 real-time"
    choice <- read input
    If choice invalid then error
    Call serveNextStudent(realTimeMode = (choice == 2))
```

```
Algorithm serveNextStudent(realTimeMode)
    currentStudent <- queue.dequeue()
    If currentStudent is null then
```

```

        Print "No students waiting"
        return
    remainingCount <- remainingCount - 1
    actualTime <- 0
    Print "Serving currentStudent.name (currentStudent.serviceType)"
    timeLeft <- currentStudent.estimatedTime

    If realTimeMode then
        Print "Presentation mode: 1 simulated minute = 1 second"
        While timeLeft > 0 do
            Print "Time left: " + timeLeft
            If skipPressed() then break
            Wait 1 second
            timeLeft <- timeLeft - 1
            actualTime <- actualTime + 1
        Else
            actualTime <- currentStudent.estimatedTime
            timeLeft <- 0

        servedCount <- servedCount + 1
        Print currentStudent.name + " completed"
        recordSummary(currentStudent, estimatedTime, actualTime)
        Print "Waiting: remainingCount | Served: servedCount | Records:
records.size()"

```

Linked List Operations (Manage service records after students are served; insert, update, search, delete)

```

Algorithm insertRecord()
    number <- read student number
    student <- findSampleStudent(number)
    If student exists then records.insertStudent(student)

Algorithm addManualStudent()
    Read student number, name, service type, estimated time
    Create new student
    queue.enqueue(student)
    records.insertStudentIfAbsent(student)
    remainingCount <- queue.size()

Algorithm updateRecordServiceType()
    number <- read student number
    newType <- read service type
    If records.updateServiceType(number, newType) then print "Updated"
    Else print "Not found"

Algorithm insertRecordAtPosition()
    number <- read student number
    student <- findSampleStudent(number)
    position <- read position
    records.insertAtPosition(student, position)

Algorithm searchRecord()

```

```

Print search options
choice <- read input
If choice = 1 then search by name
If choice = 2 then search by number
If choice = 3 then search by both
Print matches or "Not Found"

```

```

Algorithm deleteRecord()
    number <- read student number
    student <- records.searchStudent(number)
    If student not found then print "Not found"
    Else confirm deletion
        If yes then records.deleteStudent(number)

```

Sorting and Statistics (Analyze service times and compare algorithms.)

```

Algorithm sortTimes()
    times <- serviceTimes()
    Print original array
    choice <- read sort option
    Switch choice:
        1: selectionSort(times)
        2: insertionSort(times)
        3: mergeSort(times)
        4: quickSort(times)
    Print sorted array
    Perform linear and binary search for value 15
    If user wants trace then show sortWithTrace(times, choice)

```

```

Algorithm sortingExperiment()
    sizes <- [20, 50, 100, 500]
    repetitions <- 5
    For each size in sizes do
        For each algorithm (Selection, Insertion, Merge, Quick) do
            timeSort(algorithm, size, repetitions)

```

```

Algorithm timeSort(name, size, repetitions, choice)
    For run = 1 to repetitions do
        Generate random array of given size
        Start timer
        Sort array with chosen algorithm
        End timer
        Add to total
    Print average time

```

Stack and Postfix (Evaluate postfix expressions using a custom stack.)

```

Algorithm postfixDemo()
    expression <- read postfix expression
    result <- Stack.postfixEvaluation(expression)
    Print result

```

Utility(Print a report for each student served. Shows estimated vs. actual time, Highlights overtime if service exceeded estimate, Keeps accountability and daily statistics.
)

```
Algorithm recordSummary(student, estimated, actual)
    overtime <- max(0, actual - estimated)
    Print student.name, serviceType, estimated, actual, overtime
```